

Intelligent Robots Lab 6 Report

Zaoer Bot Gazebo Simulation

Prepared for submission to Blackboard

April 9, 2026

1 Task Overview

The goal of Lab 6 is to create a robot model of “myself”, add a camera and a lidar sensor, load the robot into Gazebo, complete sensor simulation, and display the sensor data in RViz. The final submission also needs to include the catkin packages and a report with code, running steps, results, and analysis.

To satisfy the task, I designed a self-inspired differential-drive robot named **zaoer_bot**. The robot uses a stylized upper-body appearance mounted on a mobile base, and includes:

- a monocular front camera publishing `/camera/image_raw` and `/camera/camera_info`,
- a 2D lidar publishing `/scan`,
- a Gazebo world with obstacles so the sensors produce visible output,
- an RViz configuration for visualizing the robot and the sensor data.

2 Package Structure

The submission contains two catkin packages.

Package / File	Purpose
<code>zaoer_bot_description</code>	Robot Xacro/URDF model
<code>zaoer_bot_gazebo</code>	Gazebo world, launch files, RViz config, capture script
<code>urdf/zaoer_bot.urdf.xacro</code>	Main robot body, wheels, camera, lidar, Gazebo plugins
<code>launch/view_zaoer_bot_lab_world.launch</code>	Start Gazebo world and spawn robot
<code>rviz/zaoer_bot_sensors.rviz</code>	RViz visualization configuration
<code>scripts/capture_lab6_results.sh</code>	Automated screenshot and topic capture

3 Robot Design

3.1 Mechanical Model

The robot is based on a differential-drive chassis because it is stable and easy to simulate. To reflect the “robot model of yourself” requirement, the body includes a torso, head, arms, chest badge, and backpack-style rear body using primitive shapes. Primitive geometry keeps the package lightweight and easy to compile.

The main physical structure is:

- `base_link`: torso and decorative visuals,
- `left_wheel_link`, `right_wheel_link`: drive wheels,
- `rear_caster_link`: rear support,
- `camera_link`: front-facing camera on the head,
- `lidar_link`: 2D lidar on the top of the head.

3.2 Sensors

Two sensor models were added in Gazebo:

- Camera sensor with `libgazebo_ros_camera.so`
- Ray-based lidar sensor with `libgazebo_ros_laser.so`

The key camera configuration is shown below.

```
1 <gazebo reference="camera_link">
2   <sensor type="camera" name="front_camera_sensor">
3     <always_on>true</always_on>
4     <update_rate>20.0</update_rate>
5     <camera name="front_camera">
6       <horizontal_fov>1.047</horizontal_fov>
7       <image>
8         <width>640</width>
9         <height>480</height>
10        <format>R8G8B8</format>
11      </image>
12    </camera>
13    <plugin name="zaoer_camera_controller"
14      filename="libgazebo_ros_camera.so">
15      <cameraName>camera</cameraName>
16      <imageTopicName>image_raw</imageTopicName>
17      <cameraInfoTopicName>camera_info</cameraInfoTopicName>
18      <frameName>camera_optical_frame</frameName>
19    </plugin>
20  </sensor>
21 </gazebo>
```

The lidar configuration is:

```
1 <gazebo reference="lidar_link">
2   <sensor type="ray" name="lidar_sensor">
3     <visualize>true</visualize>
4     <update_rate>15.0</update_rate>
5     <ray>
6       <scan>
7         <horizontal>
8           <samples>720</samples>
9           <min_angle>-1.5708</min_angle>
10          <max_angle>1.5708</max_angle>
11        </horizontal>
12      </scan>
13      <range>
14        <min>0.12</min>
15        <max>8.0</max>
16      </range>
17    </ray>
18    <plugin name="zaoer_lidar_controller"
19      filename="libgazebo_ros_laser.so">
20      <topicName>scan</topicName>
21      <frameName>lidar_link</frameName>
22    </plugin>
23  </sensor>
24 </gazebo>
```

3.3 Motion Model

The robot uses the Gazebo differential drive plugin so the mobile base publishes odometry and wheel joint states while listening to `/cmd_vel`.

```
1 <gazebo>
2   <plugin name="zaoer_diff_drive" filename="libgazebo_ros_diff_drive.so">
3     <leftJoint>left_wheel_joint</leftJoint>
4     <rightJoint>right_wheel_joint</rightJoint>
5     <wheelSeparation>0.38</wheelSeparation>
6     <wheelDiameter>0.12</wheelDiameter>
7     <commandTopic>cmd_vel</commandTopic>
8     <odometryTopic>odom</odometryTopic>
9     <robotBaseFrame>base_footprint</robotBaseFrame>
10   </plugin>
11 </gazebo>
```

4 Simulation Environment

To make the sensor output meaningful, I created a simple obstacle world instead of using a completely empty world. The world contains:

- four boundary walls,
- one orange box,
- one green cylinder,
- one purple panel.

These objects are placed in front of and around the robot so both the camera and the lidar can observe them.

5 Running Steps

5.1 Build

```
1 cd ~/catkin_ws/src
2 cp -r zaoer_bot_description zaoer_bot_gazebo .
3 cd ~/catkin_ws
4 catkin_make
5 source devel/setup.bash
```

5.2 Launch The Robot In Gazebo

```
1 roslaunch zaoer_bot_gazebo view_zaoer_bot_lab_world.launch
```

5.3 Optional Automatic Capture

```
1 bash src/zaoer_bot_gazebo/scripts/capture_lab6_results.sh ~/lab6_capture
```

6 Running Results

6.1 Gazebo Result

Figure 1 shows the robot successfully spawned in the obstacle world.

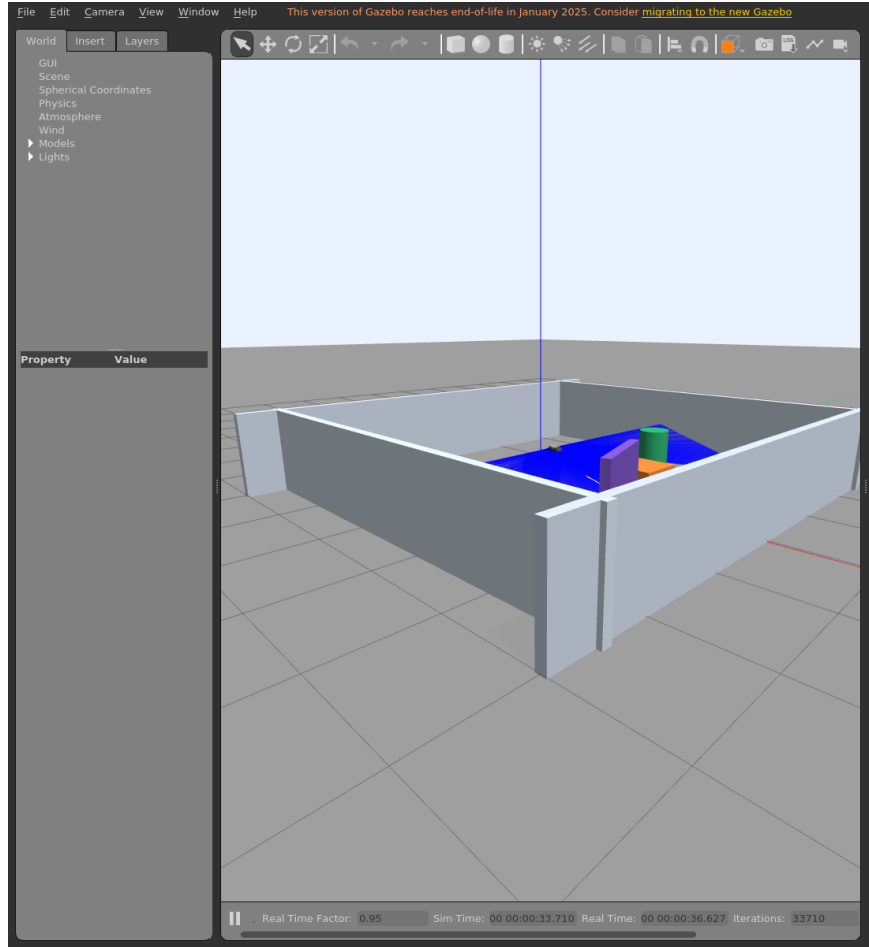


Figure 1: Gazebo simulation result

6.2 RViz Result

Figure 2 shows the RViz visualization. The upper render panel uses the camera display, while the lidar scan is overlaid as red points/lines. This confirms that the sensor data is being published and visualized in RViz.

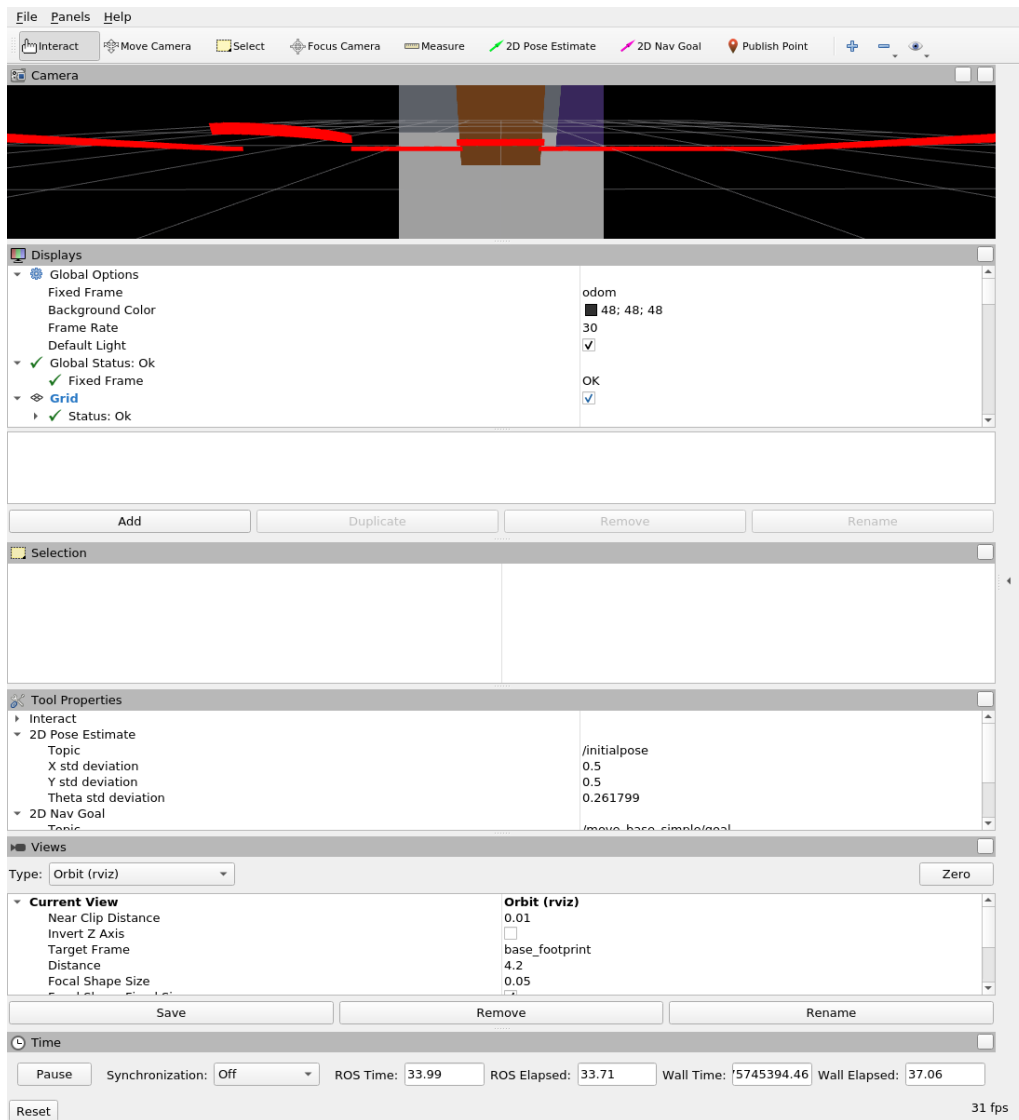


Figure 2: RViz result with sensor visualization

Figure 3 shows Gazebo and RViz running together in the same session.

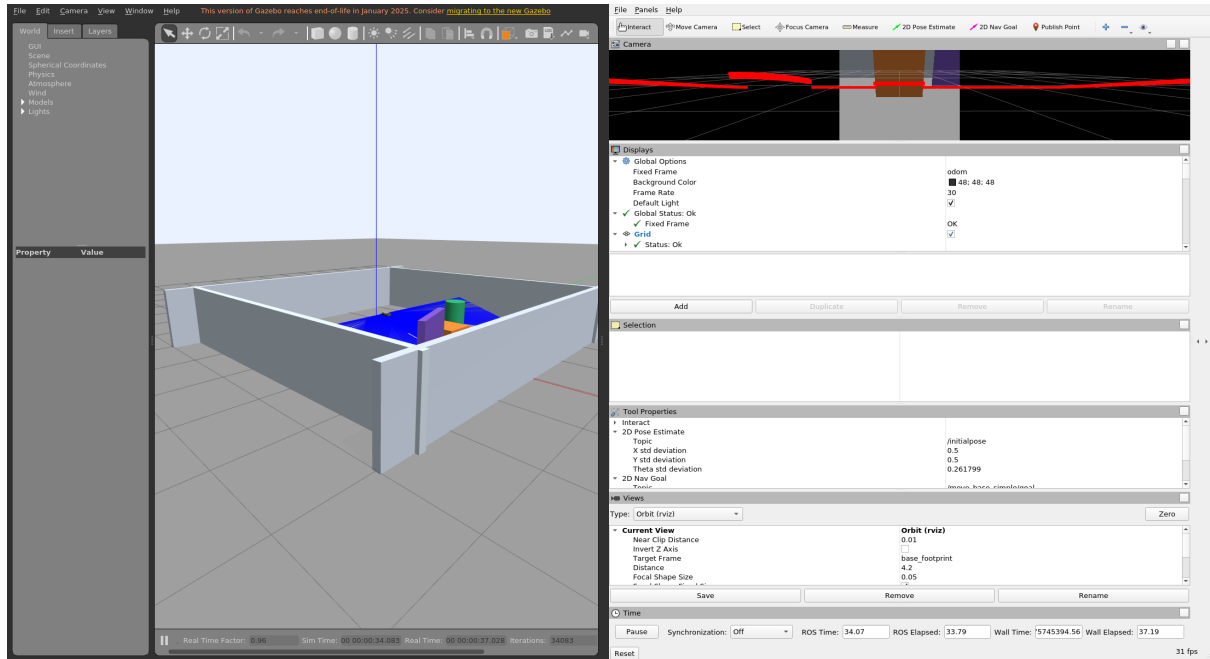


Figure 3: Combined Gazebo and RViz desktop capture

6.3 Published Topics

The main topics captured during the experiment are:

```

1 /camera/camera_info
2 /camera/image_raw
3 /clock
4 /cmd_vel
5 /joint_states
6 /odom
7 /scan
8 /tf
9 /tf_static

```

The lidar sample shows valid distance data, and the camera information topic was also published correctly.

7 Result Analysis

- The lidar simulation worked immediately in both headless and GUI modes, publishing valid `/scan` data.
- The camera plugin required a rendering context. In practice, the camera topics appeared correctly when Gazebo was launched with GUI support inside `Xvfb`. This is why the final capture script uses `Xvfb` and `gui:=true`.
- The RViz configuration successfully visualized sensor data. The camera panel and overlaid scan confirm that the robot can perceive the surrounding obstacles.

- The self-inspired robot model is lightweight because it uses boxes, cylinders, and spheres instead of external meshes. This makes the project portable and easy to rebuild on another ROS Noetic machine.

8 Conclusion

This Lab 6 submission completes all required parts:

- a self-inspired robot model was created,
- a camera and a lidar were added,
- the robot was loaded into Gazebo,
- the sensor simulation was completed,
- the sensor data was displayed in RViz,
- catkin packages and a report were prepared for submission.

The final submission folder includes the source packages, launch files, world file, RViz configuration, capture script, screenshots, and this report.